

Java Programming Lab

Coursecode:22AI471**L: P: T: S: 0:2:0:0****Exam Hours: 03****Credits: 01****CIE Marks: 50****SEE Marks: 50****Course objectives:**

1. Understand the syntax, semantics, features and applications of Java Programming Language.
2. Learn the core concepts and construct using Java.
3. Understand the advanced concept and creating java application.
4. Learn the method of creating connection between Java and DB
5. Design and build simple Java applications

Course Outcomes: At the end of the course, student will be able to:

CO1	Apply the concepts of Java to implement suitable solutions to real world problem.
CO2	Analyze the given problem to provide a Java based solution.
CO3	Design GUI application using Java Data Base Connectivity
CO4	Write clear and concise experiment reports that detail the methods, results, and conclusions of Java experiments conducted

Text Books:

1. Java The Complete Reference 9th edition, Herbert Schildt, Mc Graw Hill Education, 2014.
2. Programming with Java, E. Balagurusamy.

Reference Books:

1. J2SE Core Java, A.R.Kishore Kumar.

Website:

1. <https://www.w3schools.com/java/>
2. <https://www.javatpoint.com/java-tutorial>
3. <https://www.tutorialspoint.com/java/index.htm>
4. <https://github.com/omunite215/NPTEL-Programming-in-Java-Ultimate-Guide?tab=readme-ov-file>
5. <https://archive.nptel.ac.in/courses/106/105/106105191/#>

Sl. NO	List of Experiments	CO
1	Generate the following abnormal conditions in main(): a) NumberFormatException b) OutOfStackMemoryError (invoke main() recursively) -> can you catch this as well? c) OutOfMemoryError - heap memory -> populate heap with 1 million reachable objects. Enable a catch block for each and see whether you are able to catch the exceptions. Print the stack trace and verify.	CO1 CO4
2	Implement the this() method for cube class containing 3 constructors.	CO1 CO4
3	Develop a Java package to show dynamic polymorphism and interfaces.	CO2 CO4
4	Implement a method overriding and overloading using Java Inheritance.	CO2 CO4
5	Write a java program that implements a multi-thread application that has three threads. First thread generates random integer every 1 second and if the value is even, second thread computes the square of the number and prints. If the value is odd, the third thread will print the value of cube of the number.	CO1 CO4
6	Implement the event handling using Java AWT.	CO2 CO4
7	Develop a GUI application using Java applet.	CO3 CO4
8	Develop a GUI application using Java swings.	CO3 CO4
9	Build a GUI Java program that connects to a database using JDBC and perform Add, Delete, Display and Exit operation.	CO3 CO4
10	Create a servlet that uses Cookies to store the number of times a user has visited your servlet.	CO2 CO4

1. Generate the following abnormal conditions in main():
 - a) NumberFormatException
 - b) OutOfStackMemoryError (invoke main() recursively) -> can you catch this as well?
 - c) OutOfMemoryError - heap memory -> populate heap with 1 million reachable objects.Enable a catch block for each and see whether you are able to catch the exceptions. Print the stack trace and verify.

```
public class AbnormalConditions {

    public static void main(String[] args) {
        try {
            generateNumberFormatException();
        } catch (NumberFormatException e) {
            System.out.println("Caught NumberFormatException");
            e.printStackTrace();
        }

        try {
            generateStackOverflowError(1);
        } catch (StackOverflowError e) {
            System.out.println("Caught StackOverflowError");
            e.printStackTrace();
        }

        try {
            generateOutOfMemoryError();
        } catch (OutOfMemoryError e) {
            System.out.println("Caught OutOfMemoryError");
            e.printStackTrace();
        }
    }

    private static void generateNumberFormatException() {
        // This will throw NumberFormatException
        String invalidNumber = "abc";
        int number = Integer.parseInt(invalidNumber);
    }

    private static void generateStackOverflowError(int depth) {
        // This recursive call will eventually throw StackOverflowError
        generateStackOverflowError(depth + 1);
    }

    private static void generateOutOfMemoryError() {
        // This will throw OutOfMemoryError
        Object[] array = new Object[1000000];
        for (int i = 0; i < array.length; i++) {
            array[i] = new byte[1024 * 1024]; // 1MB each
        }
    }
}
```

2. Implement the `this()` method for `cube` class containing 3 constructors.

The `this()` method in Java is used to call one constructor from another within the same class. This can help reduce code duplication by allowing multiple constructors to reuse common initialization code. Below is an example of a `Cube` class with three constructors, demonstrating the use of the `this()` method.

```
public class Cube {
    private double sideLength;

    // Constructor 1: No arguments, default side length
    public Cube() {
        this(1.0); // Call the second constructor with default value 1.0
        System.out.println("Default constructor called");
    }

    // Constructor 2: One argument for the side length
    public Cube(double sideLength) {
        this(sideLength, true); // Call the third constructor with a boolean flag
        System.out.println("Single parameter constructor called");
    }

    // Constructor 3: Two arguments, side length and a dummy boolean flag
    public Cube(double sideLength, boolean dummyFlag) {
        this.sideLength = sideLength;
        System.out.println("Two parameter constructor called");
    }

    // Getter for side length
    public double getSideLength() {
        return sideLength;
    }

    // Method to calculate volume
    public double calculateVolume() {
        return Math.pow(sideLength, 3);
    }

    public static void main(String[] args) {
        // Test the constructors
        Cube cube1 = new Cube();
        System.out.println("Cube1 side length: " + cube1.getSideLength());
        System.out.println("Cube1 volume: " + cube1.calculateVolume());

        Cube cube2 = new Cube(3.0);
        System.out.println("Cube2 side length: " + cube2.getSideLength());
        System.out.println("Cube2 volume: " + cube2.calculateVolume());

        Cube cube3 = new Cube(4.0, true);
        System.out.println("Cube3 side length: " + cube3.getSideLength());
        System.out.println("Cube3 volume: " + cube3.calculateVolume());
    }
}
```

3. Develop a Java package to show dynamic polymorphism and interfaces.

To demonstrate dynamic polymorphism and interfaces in Java, we can create a package called **shapes** that includes an interface **Shape** and multiple classes implementing this interface. We will then show how dynamic polymorphism works by using a base class reference to call overridden methods of derived classes.

- **Package Structure:**

```
shapes/  
Shape.java  
Circle.java  
Rectangle.java  
Main.java
```

- **Interface Shape**

First, we define the **Shape** interface with a method to calculate the area:

```
package shapes;  
public interface Shape { double calculateArea(); }
```

- **Class Circle**

Next, we implement the **Shape** interface in the **Circle** class:

```
package shapes;  
public class Circle implements Shape {  
    private double radius;  
    public Circle(double radius) {  
        this.radius = radius;  
    }  
    @Override  
    public double calculateArea() {  
        return Math.PI * radius * radius;  
    }  
    public double getRadius() {  
        return radius;  
    }  
}
```

- **Class Rectangle**

Similarly, we implement the **Shape** interface in the **Rectangle** class:

```
package shapes;  
  
public class Rectangle implements Shape {  
    private double width;  
    private double height;  
  
    public Rectangle(double width, double height) {  
        this.width = width;  
        this.height = height;  
    }  
    @Override  
    public double calculateArea() {  
        return width * height;  
    }  
    public double getWidth() {  
        return width;  
    }  
    public double getHeight() {
```

```
        return height;
    }
}
```

- **Main Class to Demonstrate Dynamic Polymorphism**

In the **Main** class, we demonstrate dynamic polymorphism by using an array of **Shape** references to call the **calculateArea** method:

```
package shapes;

public class Main {
    public static void main(String[] args) {
        Shape[] shapes = new Shape[3];
        shapes[0] = new Circle(5.0);
        shapes[1] = new Rectangle(4.0, 6.0);
        shapes[2] = new Circle(2.5);

        for (Shape shape : shapes) {
            System.out.println("Area: " + shape.calculateArea());
        }
    }
}
```

Explanation

1. **Interface Shape:**

- Declares the **calculateArea** method, which must be implemented by any class that implements this interface.

2. **Class Circle:**

- Implements the **Shape** interface and provides the implementation for the **calculateArea** method.
- The area of a circle is calculated using the formula πr^2 .

3. **Class Rectangle:**

- Implements the **Shape** interface and provides the implementation for the **calculateArea** method.
- The area of a rectangle is calculated using the formula $\text{width} \times \text{height}$.

4. **Class Main:**

- Demonstrates dynamic polymorphism by creating an array of **Shape** references.
- Each element in the array is an instance of a class that implements the **Shape** interface (**Circle** or **Rectangle**).
- The **calculateArea** method is called on each element in the array. At runtime, the JVM determines the appropriate method implementation to call based on the actual object type (either **Circle** or **Rectangle**).

Running the Program

To compile and run the program, you can use the following commands in the terminal:

```
javac shapes/*.java
java shapes.Main
```

This will produce the following output:

```
Area: 78.53981633974483
Area: 24.0
Area: 19.634954084936208
```

4, Implement a method overriding and overloading using Java Inheritance.

To demonstrate method overriding and overloading using Java inheritance, we can create a base class **Animal** and a derived class **Dog**. We'll override a method from the base class in the derived class and also overload a method in the derived class.

Class Structure

- **Animal**: The base class.
- **Dog**: The derived class that extends **Animal**.

1. **Animal.java**

First, let's define the base class **Animal** with a method that we'll override and a method that will be overloaded in the derived class.

```
public class Animal {  
    public void makeSound() {  
        System.out.println("The animal makes a sound");  
    }  
  
    public void eat(String food) {  
        System.out.println("The animal eats " + food);  
    }  
}
```

2. **Dog.java**

Next, we define the derived class **Dog** that overrides the **makeSound** method and overloads the **eat** method.

```
public class Dog extends Animal {  
    // Overriding the makeSound method  
    @Override  
    public void makeSound() {  
        System.out.println("The dog barks");  
    }  
  
    // Overloading the eat method  
    public void eat(String food, int quantity) {  
        System.out.println("The dog eats " + quantity + " units of " + food);  
    }  
}
```

3. **Main.java**

Finally, we'll create a **Main** class to test method overriding and overloading.

```
public class Main {  
    public static void main(String[] args) {  
        Animal myAnimal = new Animal();  
        Dog myDog = new Dog();  
  
        // Demonstrate method overriding  
        myAnimal.makeSound(); // Should print: The animal makes a sound  
        myDog.makeSound();    // Should print: The dog barks  
    }  
}
```

```
// Demonstrate method overloading
myAnimal.eat("food");    // Should print: The animal eats food
myDog.eat("bone");      // Should print: The animal eats bone (inherited method)
myDog.eat("bone", 2);    // Should print: The dog eats 2 units of bone (overloaded method)
}
}
```

Explanation

1. Class **Animal**:

- Contains a method **makeSound** which prints a general message.
- Contains a method **eat** which takes a **String** parameter and prints what the animal eats.

2. Class **Dog**:

- Extends the **Animal** class.
- Overrides the **makeSound** method to provide a specific implementation for a dog.
- Overloads the **eat** method by adding a version that takes an additional **int** parameter to specify the quantity.

3. Class **Main**:

- Creates instances of **Animal** and **Dog**.
- Demonstrates method overriding by calling **makeSound** on both instances.
- Demonstrates method overloading by calling both versions of the **eat** method on the **Dog** instance.

Running the Program

To compile and run the program, you can use the following commands in the terminal:

```
javac Animal.java Dog.java Main.java
```

```
java Main
```

This will produce the following output:

```
The animal makes a sound
The dog barks
The animal eats food
The animal eats bone
The dog eats 2 units of bone
```


5. Write a java program that implements a multi-thread application that has three threads. First thread generates random integer every 1 second and if the value is even, second thread computes the square of the number and prints. If the value is odd, the third thread will print the value of cube of the number.

```
import java.util.Random;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        SharedNumber sharedNumber = new SharedNumber();
```

```
        Thread generatorThread = new Thread(new NumberGenerator(sharedNumber));
```

```
        Thread squareThread = new Thread(new SquareCalculator(sharedNumber));
```

```
        Thread cubeThread = new Thread(new CubeCalculator(sharedNumber));
```

```
        generatorThread.start();
```

```
        squareThread.start();
```

```
        cubeThread.start();
```

```
    }
```

```
}
```

```
class SharedNumber {
```

```
    private int number;
```

```
    private boolean newNumberGenerated = false;
```

```
    public synchronized void setNumber(int number) {
```

```
        this.number = number;
```

```
        this.newNumberGenerated = true;
```

```
        notifyAll();
```

```
    }
```

```
    public synchronized int getNumber() throws InterruptedException {
```

```
        while (!newNumberGenerated) {
```

```
            wait();
```

```
        }
```

```
        newNumberGenerated = false;
```

```
        return number;
```

```
    }
```

```
}
```

```
class NumberGenerator implements Runnable {
```

```
    private SharedNumber sharedNumber;
```

```
    private Random random = new Random();
```

```
    public NumberGenerator(SharedNumber sharedNumber) {
```

```
        this.sharedNumber = sharedNumber;
```

```
    }
```

```
    @Override
```

```
    public void run() {
```

```
        while (true) {
```

```
        int number = random.nextInt(100);
        System.out.println("Generated number: " + number);
        sharedNumber.setNumber(number);
        try {
            Thread.sleep(1000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

```
class SquareCalculator implements Runnable {
    private SharedNumber sharedNumber;

    public SquareCalculator(SharedNumber sharedNumber) {
        this.sharedNumber = sharedNumber;
    }

    @Override
    public void run() {
        while (true) {
            try {
                int number = sharedNumber.getNumber();
                if (number % 2 == 0) {
                    System.out.println("Square of " + number + " is " + (number * number));
                }
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}
```

```
class CubeCalculator implements Runnable {
    private SharedNumber sharedNumber;

    public CubeCalculator(SharedNumber sharedNumber) {
        this.sharedNumber = sharedNumber;
    }

    @Override
    public void run() {
        while (true) {
            try {
                int number = sharedNumber.getNumber();
                if (number % 2 != 0) {
                    System.out.println("Cube of " + number + " is " + (number * number * number));
                }
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}
```

```
    }  
    }  
}
```

Explanation**1. SharedNumber Class:**

- This class holds the generated number and provides synchronized methods to set and get the number, ensuring proper communication between threads.

2. NumberGenerator Class:

- Implements **Runnable** and generates a random number every 1 second.
- Sets the generated number in the **SharedNumber** instance.

3. SquareCalculator Class:

- Implements **Runnable** and waits for a number to be set.
- If the number is even, it calculates and prints the square of the number.

4. CubeCalculator Class:

- Implements **Runnable** and waits for a number to be set.
- If the number is odd, it calculates and prints the cube of the number.

Running the Program

To compile and run the program, you can use the following commands in the terminal:

```
javac Main.java
```

```
java Main
```

6. Implement the event handling using Java AWT.

```
import java.awt.*;
import java.awt.event.*;

public class SimpleAWTEventHandling extends Frame implements ActionListener {
    private Button button;
    private Label label;

    public SimpleAWTEventHandling() {
        // Set the frame properties
        setTitle("Simple AWT Event Handling Example");
        setSize(300, 200);
        setLayout(new FlowLayout());
        setVisible(true);

        // Create a button
        button = new Button("Click Me");
        button.addActionListener(this); // Register the event listener

        // Create a label
        label = new Label("Button not clicked yet.");

        // Add button and label to the frame
        add(button);
        add(label);

        // Add window closing event
        addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent windowEvent) {
                System.exit(0);
            }
        });
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        // Update the label when the button is clicked
        label.setText("Button clicked!");
    }

    public static void main(String[] args) {
        new SimpleAWTEventHandling();
    }
}
```

Explanation

- SimpleAWTEventHandling Class:**
 - Extends **Frame** to create a main window.
 - Implements **ActionListener** to handle button click events.
- Constructor:**
 - Sets the frame title, size, and layout.
 - Makes the frame visible.

- Creates a **Button** and a **Label**.
 - Registers the current instance (**this**) as an **ActionListener** for the button.
 - Adds the button and label to the frame.
 - Adds a window listener to handle the window close event.
3. **actionPerformed Method:**
- This method is called when the button is clicked.
 - Updates the label text to indicate that the button has been clicked.
4. **Main Method:**
- Creates an instance of the **SimpleAWTEventHandling** class, which sets up and displays the GUI.

Running the Program

To compile and run the program, use the following commands in the terminal:

```
javac SimpleAWTEventHandling.java  
java SimpleAWTEventHandling
```

7. Develop a GUI application using Java applet.

Step-by-Step Implementation

Step 1: Create the Applet Class

Create a Java file named SimpleApplet.java:

```
import java.applet.Applet;
import java.awt.*;
import java.awt.event.*;

public class SimpleApplet extends Applet implements ActionListener {
    private Button button;
    private Label label;

    @Override
    public void init() {
        // Set the layout
        setLayout(new FlowLayout());

        // Create a button
        button = new Button("Click Me");
        button.addActionListener(this); // Register the event listener

        // Create a label
        label = new Label("Button not clicked yet.");

        // Add button and label to the applet
        add(button);
        add(label);
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        // Update the label when the button is clicked
        label.setText("Button clicked!");
    }
}
```

Explanation

1. **SimpleApplet Class:**
 - Extends **Applet** to create an applet.
 - Implements **ActionListener** to handle button click events.
2. **init Method:**
 - This method is called to initialize the applet.
 - Sets the layout to **FlowLayout**.
 - Creates a **Button** and a **Label**.
 - Registers the current instance (**this**) as an **ActionListener** for the button.
 - Adds the button and label to the applet.
3. **actionPerformed Method:**
 - This method is called when the button is clicked.
 - Updates the label text to indicate that the button has been clicked.

HTML File to Run the AppletTo run the applet, you need to create an HTML file. Create a file named **SimpleApplet.html**:

```
<!DOCTYPE html>
<html>
<head>
  <title>Simple Applet Example</title>
</head>
<body>
  <h1>Simple Applet Example</h1>
  <applet code="SimpleApplet.class" width="300" height="200"></applet>
</body>
</html>
```

Running the Applet

To compile and run the applet, follow these steps:

1. **Compile the Applet:**
javac SimpleApplet.java
2. **Run the Applet:** Use an applet viewer or an older browser that still supports applets. For example, you can use the **appletviewer** tool that comes with the JDK to view the applet:
appletviewer SimpleApplet.html

Alternative: Using JavaFX for GUI Applications

For creating modern Java GUI applications, JavaFX is the recommended approach. JavaFX provides a rich set of GUI components and better support for modern development practices.

Step 1: Create the Main Class

Create a Java file named **SimpleJavaFXApp.java**:

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class SimpleJavaFXApp extends Application {
    @Override
    public void start(Stage primaryStage) {
        // Create a label
        Label label = new Label("Button not clicked yet.");

        // Create a button
        Button button = new Button("Click Me");
        button.setOnAction(e -> label.setText("Button clicked!"));

        // Create a layout and add the button and label
        VBox layout = new VBox(10);
        layout.getChildren().addAll(button, label);

        // Create a scene and add it to the stage
        Scene scene = new Scene(layout, 300, 200);
        primaryStage.setScene(scene);
        primaryStage.setTitle("Simple JavaFX App");
    }
}
```

```
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Running the JavaFX Application

To compile and run the JavaFX application, use the following commands:

1. **Compile the JavaFX Application:**

```
javac --module-path /path/to/javafx-sdk-15.0.1/lib --add-modules javafx.controls
SimpleJavaFXApp.java
```

2. **Run the JavaFX Application:**

```
java --module-path /path/to/javafx-sdk-15.0.1/lib --add-modules javafx.controls
SimpleJavaFXApp
```

Replace **/path/to/javafx-sdk-15.0.1/lib** with the actual path to your JavaFX SDK.

This JavaFX example provides a modern alternative to Java applets for creating GUI applications.

8. Develop a GUI application using Java swings.

Step-by-Step Implementation

Step 1: Create the Main Class

Create a Java file named SimpleSwingApp.java:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class SimpleSwingApp {
    public static void main(String[] args) {
        // Create the frame
        JFrame frame = new JFrame("Simple Swing App");
        frame.setSize(300, 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setLayout(new FlowLayout());

        // Create a button
        JButton button = new JButton("Click Me");

        // Create a label
        JLabel label = new JLabel("Button not clicked yet.");

        // Add an action listener to the button
        button.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                label.setText("Button clicked!");
            }
        });

        // Add button and label to the frame
        frame.add(button);
        frame.add(label);

        // Set the frame visibility
        frame.setVisible(true);
    }
}
```

Explanation

1. **JFrame:** The main window of the application. It's set up with a title "Simple Swing App", a size of 300x200 pixels, and a default close operation to exit the application when the window is closed.
2. **JButton:** A button labeled "Click Me". When clicked, it triggers an action event.
3. **JLabel:** A label displaying "Button not clicked yet.". This label's text will be updated when the button is clicked.
4. **ActionListener:** An anonymous inner class is used to handle button click events. When the button is clicked, the **actionPerformed** method is called, updating the label's text to "Button clicked!".

Running the Program

To compile and run the program, use the following commands in the terminal:

1. **Compile the Java file:**
`javac SimpleSwingApp.java`
2. Run the compiled Java program:
`java SimpleSwingApp`

Expected Behavior

1. A window with the title "Simple Swing App" will appear.
2. The window contains a button labeled "Click Me" and a label displaying "Button not clicked yet."
3. When the button is clicked, the label text changes to "Button clicked!"

9. Build a GUI Java program that connects to a database using JDBC and perform Add, Delete, Display and Exit operation.
 1. Set up the database.
 2. Create the JDBC connection.
 3. Design the GUI using Swing.
 4. Implement Add, Delete, Display, and Exit operations.

Step 1: Set Up the Database

For this example, we will use an SQLite database. SQLite is a lightweight database that is easy to set up and use.

First, you need to download the SQLite JDBC driver. You can find it [here](#). Add the downloaded JAR file to your project classpath.

Create a database file (e.g., example.db) and a table people with the following schema:

```
CREATE TABLE people (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  name TEXT NOT NULL,  
  age INTEGER NOT NULL  
);
```

Step 2: Create the JDBC Connection

Create a Java class to handle the JDBC connection.

```
import java.sql.Connection;  
import java.sql.DriverManager;  
import java.sql.SQLException;  
  
public class DatabaseConnection {  
  private static final String URL = "jdbc:sqlite:example.db";  
  
  public static Connection getConnection() throws SQLException {  
    return DriverManager.getConnection(URL);  
  }  
}
```

Step 3: Design the GUI using Swing

Create the main GUI class **DatabaseGUI.java**.

```
import javax.swing.*;  
import java.awt.*;  
import java.awt.event.ActionEvent;  
import java.awt.event.ActionListener;  
import java.sql.*;  
  
public class DatabaseGUI extends JFrame {  
  private JTextField nameField;  
  private JTextField ageField;  
  private JTextArea displayArea;  
  
  public DatabaseGUI() {  
    setTitle("Database Operations");  
    setSize(400, 300);  
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
  }  
}
```

```
setLayout(new FlowLayout());

// Name input
add(new JLabel("Name:"));
nameField = new JTextField(20);
add(nameField);

// Age input
add(new JLabel("Age:"));
ageField = new JTextField(3);
add(ageField);

// Buttons
JButton addButton = new JButton("Add");
addButton.addActionListener(new AddButtonListener());
add(addButton);

JButton deleteButton = new JButton("Delete");
deleteButton.addActionListener(new DeleteButtonListener());
add(deleteButton);

JButton displayButton = new JButton("Display");
displayButton.addActionListener(new DisplayButtonListener());
add(displayButton);

JButton exitButton = new JButton("Exit");
exitButton.addActionListener(e -> System.exit(0));
add(exitButton);

// Display area
displayArea = new JTextArea(10, 30);
add(new JScrollPane(displayArea));

setVisible(true);
}

private class AddButtonListener implements ActionListener {
    public void actionPerformed(ActionEvent e) {
        String name = nameField.getText();
        int age = Integer.parseInt(ageField.getText());

        try (Connection conn = DatabaseConnection.getConnection();
            PreparedStatement stmt = conn.prepareStatement("INSERT INTO people (name, age)
VALUES (?, ?)")) {
            stmt.setString(1, name);
            stmt.setInt(2, age);
            stmt.executeUpdate();
            displayArea.setText("Record added.");
        } catch (SQLException ex) {
            displayArea.setText("Error: " + ex.getMessage());
        }
    }
}
```

```
    }  
}  
  
private class DeleteButtonListener implements ActionListener {  
    public void actionPerformed(ActionEvent e) {  
        String name = nameField.getText();  
  
        try (Connection conn = DatabaseConnection.getConnection();  
            PreparedStatement stmt = conn.prepareStatement("DELETE FROM people WHERE name =  
?")) {  
            stmt.setString(1, name);  
            int rows = stmt.executeUpdate();  
            displayArea.setText(rows > 0 ? "Record deleted." : "No such record.");  
        } catch (SQLException ex) {  
            displayArea.setText("Error: " + ex.getMessage());  
        }  
    }  
}  
  
private class DisplayButtonListener implements ActionListener {  
    public void actionPerformed(ActionEvent e) {  
        try (Connection conn = DatabaseConnection.getConnection();  
            Statement stmt = conn.createStatement();  
            ResultSet rs = stmt.executeQuery("SELECT * FROM people")) {  
  
            StringBuilder results = new StringBuilder();  
            while (rs.next()) {  
                results.append(rs.getInt("id")).append(": ")  
                    .append(rs.getString("name")).append(", ")  
                    .append(rs.getInt("age")).append("\n");  
            }  
            displayArea.setText(results.toString());  
        } catch (SQLException ex) {  
            displayArea.setText("Error: " + ex.getMessage());  
        }  
    }  
}  
  
public static void main(String[] args) {  
    new DatabaseGUI();  
}
```

Explanation

1. **DatabaseConnection Class:** This class handles the database connection using SQLite.
2. **DatabaseGUI Class:**
 - Sets up the main window with input fields for **name** and **age**, buttons for **Add**, **Delete**, **Display**, and **Exit**, and a text area for displaying results.
 - **AddButtonListener:** Inserts a new record into the database.
 - **DeleteButtonListener:** Deletes a record from the database based on the name.
 - **DisplayButtonListener:** Displays all records from the database.
3. **main Method:** Creates an instance of **DatabaseGUI**, which sets up and displays the GUI.

Running the Program

To compile and run the program, ensure that the SQLite JDBC driver is in your classpath, and use the following commands in the terminal:

1. **Compile the Java files:**

```
javac -cp .:sqlite-jdbc-<version>.jar DatabaseConnection.java DatabaseGUI.java
```

2. **Run the compiled Java program:**

```
java -cp .:sqlite-jdbc-<version>.jar DatabaseGUI
```

Replace **<version>** with the actual version of the SQLite JDBC driver you downloaded.

10. Create a servlet that uses Cookies to store the number of times a user has visited your servlet.

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class VisitCounterServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        // Get the existing cookie
        Cookie[] cookies = request.getCookies();
        int visitCount = 0;

        if (cookies != null) {
            for (Cookie cookie : cookies) {
                if (cookie.getName().equals("visitCount")) {
                    try {
                        visitCount = Integer.parseInt(cookie.getValue());
                        visitCount++; // Increment the visit count
                    } catch (NumberFormatException e) {
                        // Invalid cookie value, reset the count
                        visitCount = 1;
                    }
                    break;
                }
            }
        } else {
            // No cookies found, set the visit count to 1
            visitCount = 1;
        }

        // Create or update the cookie
        Cookie visitCookie = new Cookie("visitCount", String.valueOf(visitCount));
        visitCookie.setMaxAge(60 * 60 * 24 * 30); // Cookie expires in 30 days
        response.addCookie(visitCookie);

        // Set the response content type
        response.setContentType("text/html");

        // Write the response
        PrintWriter out = response.getWriter();
        out.println("<html><head><title>Visit Counter Servlet</title></head><body>");
        out.println("<h2>Welcome to the Visit Counter Servlet</h2>");
        out.println("<p>You have visited this page " + visitCount + " time(s).</p>");
        out.println("</body></html>");
    }
}
```

Explanation:

1. The **doGet** method of the servlet is responsible for handling HTTP GET requests.

2. It first retrieves the existing cookies from the request using **request.getCookies()**.
3. If the cookies contain a cookie named "visitCount", it parses its value to get the current visit count and increments it.
4. If the cookie is not found or its value is invalid, it sets the visit count to 1.
5. It then creates a new cookie named "visitCount" with the updated visit count and adds it to the response using **response.addCookie()**.
6. Finally, it generates an HTML response that displays the current visit count to the user.

Deployment:

To deploy this servlet, you need to follow these steps:

1. Compile the servlet code and package it into a WAR file.
2. Deploy the WAR file to a servlet container such as Apache Tomcat.
3. Access the servlet using its URL in a web browser.

When a user accesses the servlet URL, the servlet will set a cookie in the user's browser to track the number of times the servlet has been visited. On subsequent visits, the servlet will retrieve the cookie value, increment it, and update the cookie with the new value.

PREREQUISITE LAB PROGRAM:

Lab1	a)Write a program to print all data types values on the screen.
	b)WAP that convert string to character using toString() and valueOf().
Lab2	a)Write an application that accepts one command line argument, display the line of reporting if number is even or odd.
	b)Write a program to find out the array index or position where sum of numbers preceding the index is equals to sum of numbers succeeding the index.
Lab3	a)Using the concept of method overloading Write method for calculating the area of triangle ,circle and rectangle.
	b)WAP that creates a class circle with instance variables for the center and the radius. Initialize and display its variables.
Lab4	a) Modify experiment lab3 (b) to have a constructor in class circle to initialize its variables.
	b)WAP that implements method overriding.
Lab5	a) WAP to illustrate simple and multilevel inheritance.
	b) WAP illustrating all uses of super keywords.
Lab6	a) Create an abstract class shape. Let rectangle and triangle inherit this shape class. Add necessary functions.
	b) WAP to create a simple package.
Lab7	a) Implement an application that creates an Java interface.
	b) Write an application that shows how to create a user-defined exception.
Lab8	a) Create a customized exception and also make use of all the 5 exception keywords.
	b) Write an Applet that displays <Hello World= (Background color-black, text color-blue and your name in the statuswindow.)
Lab9	a)Write a java program to show multithreaded producer and consumer application.
	b) WAP that displays your name whenever the mouse is clicked.
Lab10	a) Write a program that read from a file and write to file.
	b) Create a simple login table and establish JDBC connection using java program.
Lab11	a) Create runnable jar file in java.
	b) Display image on a button using swing.
Lab12	Write a servlet which displays current system date and time.
	Write a program that handle HTTP request and HTTP response.

1a) Write a program to print all data types values on the screen.

// Java Program to Demonstrate Char Primitive Data Type

// Class

class GFG {

 // Main driver method

 public static void main(String args[])
 {

 // Creating and initializing custom character

 char a = 'G';

 // Integer data type is generally

 // used for numeric values

 int i = 89;

 // use byte and short

 // if memory is a constraint

 byte b = 4;

 // this will give error as number is

 // larger than byte range

 // byte b1 = 7888888955;

 short s = 56;

 // this will give error as number is

 // larger than short range

 // short s1 = 87878787878;

 // by default fraction value

 // is double in java

 double d = 4.355453532;

 // for float use 'f' as suffix as standard

 float f = 4.7333434f;

 // need to hold big range of numbers then we need

 // this data type

 long l = 12121;

 System.out.println("char: " + a);

 System.out.println("integer: " + i);

 System.out.println("byte: " + b);

 System.out.println("short: " + s);

 System.out.println("float: " + f);

 System.out.println("double: " + d);

 System.out.println("long: " + l);

 }

}

1b) WAP that convert string to character using toString() and valueOf()..

Java Convert char to String

We can convert char to String in java using String.valueOf(char) method of String class and Character.toString(char) method of Character class.

// Java Program to Convert Char to String

// Using toString() method of Character class

// Importing the required packages

import java.io.*;

import java.util.*;

// Main class

class GFG {

 // Main driver method

 public static void main(String[] args)

 {

 // Declaring a char variable

 char c = 'G';

 // Converting the char to String using toString

 // method of Character class and storing it in a string

 String s = Character.toString(c);

 // Print and display the above string

 System.out.println("Char to String using Character.toString():" + " " + s);

 }

}

// Java Program to Convert Char to String

// Using valueOf() method of String class

// Importing the required packages

import java.io.*;

import java.util.*;

// Main class

class GFG {

 // Main driver method

 public static void main(String[] args)

 {

 // Declaring a char variable

 char c = 'G';

 // Converting char to String

 // using String.valueOf() method

 String s = String.valueOf(c);

 // Print and display the String s

 System.out.println("Char to String using String.valueOf() method :"+ " " + s);

 }

}

2a) Write an application that accepts one command line argument, display the line of reporting if number is even or odd.

// Java program to check

// if a number is even or odd

// using command line arguments

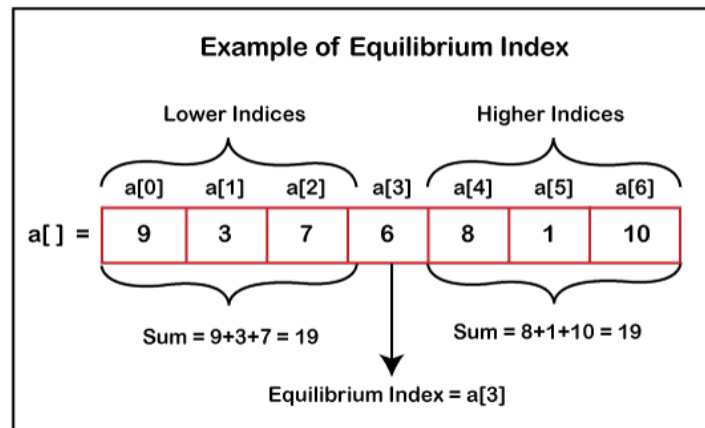
```
Public class GFG {  
    // Function to the check Even or Odd  
    public static int isEvenOrOdd(int num)  
    {  
        return (num % 2);  
    }  
    // Driver code  
    public static void main(String[] args)  
    {  
        // Check if length of args array is  
        // greater than 0  
        if (args.length > 0) {  
            // Get the command line argument and Convert it from string type to integer type  
            int num = Integer.parseInt(args[0]);  
            // Get the command line argument and check if it is even or odd  
            int res = isEvenOrOdd(num);  
            // Check if res is 0 or 1  
            if (res == 0)  
                // Print Even  
                System.out.println("Even\n");  
            else  
                // Print Odd  
                System.out.println("Odd\n");  
        }  
        else  
            System.out.println("No command line " + "arguments found.");  
    }  
}
```

Or

import java.util.Scanner;

```
public class EvenOdd {  
    public static void main(String[] args) {  
        Scanner reader = new Scanner(System.in);  
        System.out.print("Enter a number: ");  
        int num = reader.nextInt();  
        if(num % 2 == 0)  
            System.out.println(num + " is even");  
        else  
            System.out.println(num + " is odd");  
    }  
}
```

2b) Write a program to find out the array index or position where sum of numbers preceding the index is equals to sum of numbers succeeding the index.



```

public class EqualSumIndex {
    public static int findEqualSumIndex(int[] nums) {
        // Edge case: if array is empty, return -1
        if(nums == null || nums.length == 0)
            return -1;

        int n = nums.length;
        // Calculate the total sum of the array
        int totalSum = 0;
        for (int num : nums) {
            totalSum += num;
        }

        // Initialize sum of numbers preceding the index
        int leftSum = 0;

        // Iterate through the array to find the index
        for (int i = 0; i < n; i++) {
            // Subtract the current element from totalSum
            totalSum -= nums[i];

            // If sum of numbers preceding the index is equal to sum of numbers succeeding the index,
            // return the current index
            if (leftSum == totalSum)
                return i;

            // Update the sum of numbers preceding the index
            leftSum += nums[i];
        }

        // If no such index is found, return -1
        return -1;
    }

    public static void main(String[] args) {
        int[] nums = {1, 2, 3, 4, 3, 2, 1};
        int index = findEqualSumIndex(nums);
    }
}

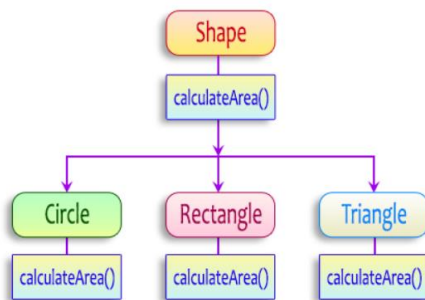
```

```
        if (index != -1) {
            System.out.println("Index where the sum of numbers preceding the index equals the sum of
numbers succeeding the index: " + index);
        } else {
            System.out.println("No such index found.");
        }
    }
}
```

Or

```
public class EquilibriumIndexExample1
{
    //function to find the equilibrium index
    static int eqbmindex(int array[], int n)
    {
        int i, j;
        //lsum denotes the left sum and rsum denotes the right sum
        int lsum, rsum;
        for (i = 0; i < n; ++i)
        {
            lsum = 0;
            //finds the sum upto current index (exclude)
            for (j = 0; j < i; j++)
                lsum = lsum + array[j];
            rsum = 0;
            //finds the sum from current index to last index (exclude)
            for (j = i + 1; j < n; j++)
                rsum = rsum + array[j];
            //if rsum and lsum are equal, returns i (equilibrium index), else returns -1
            if (lsum == rsum)
                return i;
        }
        return -1;
    }
    public static void main(String args[])
    {
        int array[] = {9, 3, 7, 6, 8, 1, 10};
        int arraysize = array.length;
        System.out.print("Equilibrium Index is: ");
        System.out.println(eqbmindex(array, arraysize));
    }
}
```

3a) Using the concept of **method overloading** Write method for calculating the area of triangle ,circle and rectangle.



```

class Shape{
    void calculateArea(float x)
    {
        System.out.println("Area of the square: "+x*x+" sq units");
    }
    void calculateArea(float x, float y)
    {
        System.out.println("Area of the rectangle: "+x*y+" sq units");
    }
    void calculateArea(double r)
    {
        double area = 3.14*r*r;
        System.out.println("Area of the circle: "+area+" sq units");
    }
    void calculateArea(double h, double b)
    {
        double area =0.5*h*b;
        System.out.println("Area of the triangle: "+area+" sq units");
    }
    public static void main(String args[]){
        Shape obj = new Shape();

        /* This statement will call the first area() method because we are passing only one argument with
        the "f" suffix. f is used to denote the float numbers */
        obj.calculateArea(6.1f);

        /* This will call the second method because we are passing two arguments and only second
        method has two arguments */
        obj.calculateArea(10,22);

        /* This will call the second method because we have not suffixed
        * the value with "f" when we do not suffix a float value with f
        * then it is considered as type double.
        */
        obj.calculateArea(6.1);
        obj.calculateArea(16.5,26.1);

    }
}
  
```

3b) Write a program that creates a class circle with instance variables for the center and the radius. Initialize and display its variables.

The **this keyword** refers to the current object in a method or constructor.

```
public class Circle {
    // Instance variables for the center and radius
    private double centerX;
    private double centerY;
    private double radius;

    // Constructor to initialize the center and radius
    public Circle(double centerX, double centerY, double radius) {
        this.centerX = centerX;
        this.centerY = centerY;
        this.radius = radius;
    }

    // Method to display the circle's variables
    public void display() {
        System.out.println("Center: (" + centerX + ", " + centerY + ")");
        System.out.println("Radius: " + radius);
    }

    public static void main(String[] args) {
        // Create a Circle object and initialize its variables
        Circle circle = new Circle(0.0, 0.0, 5.0);

        // Display the circle's variables
        System.out.println("Circle Information:");
        circle.display();
    }
}
```


4a) Modify experiment lab3 (b) to have a constructor in class circle to initialize its variables.

```
public class Circle {
    private double[] center;
    private double radius;

    // Constructor to initialize variables
    public Circle(double[] center, double radius) {
        this.center = center;
        this.radius = radius;
    }

    // Method to display circle attributes
    public void display() {
        System.out.println("Center coordinates: (" + center[0] + ", " + center[1] + ")");
        System.out.println("Radius: " + radius);
    }

    public static void main(String[] args) {
        // Example values
        double[] center = {0, 0};
        double radius = 5;

        // Create an instance of Circle
        Circle circle = new Circle(center, radius);

        // Display circle attributes
        circle.display();
    }
}
```

4b) Write a program that implements method overriding.

Method overriding occurs when a subclass provides a specific implementation of a method that is already provided by one of its superclasses.

- We have a superclass **Animal** with a method **makeSound()**.
- The **Dog** and **Cat** classes are subclasses of **Animal** that override the **makeSound()** method.
- In the **main** method, we create instances of **Animal**, **Dog**, and **Cat**. When we call the **makeSound()** method on these instances, the specific implementations from the subclasses are invoked due to method overriding.

```
class Animal {
    public void makeSound() {
        System.out.println("Animal makes a sound");
    }
}
class Dog extends Animal {
    @Override
    public void makeSound() {
        System.out.println("Dog barks");
    }
}
class Cat extends Animal {
    @Override
    public void makeSound() {
        System.out.println("Cat meows");
    }
}

public class MethodOverridingExample {
    public static void main(String[] args) {
        Animal animal1 = new Animal();
        Animal animal2 = new Dog();
        Animal animal3 = new Cat();

        animal1.makeSound(); // Output: Animal makes a sound
        animal2.makeSound(); // Output: Dog barks
        animal3.makeSound(); // Output: Cat meows
    }
}
```

5a) Write a program to illustrate simple and multilevel inheritance.

```
// Parent class
class Animal {
    public void eat() {
        System.out.println("Animal is eating");
    }
}

// Child class inheriting from Animal
class Dog extends Animal {
    public void bark() {
        System.out.println("Dog is barking");
    }
}

// Grandchild class inheriting from Dog
class Labrador extends Dog {
    public void display() {
        System.out.println("Labrador is a type of dog");
    }
}

public class InheritanceExample {
    public static void main(String[] args) {
        // Simple inheritance example
        Dog myDog = new Dog();
        myDog.eat(); // Output: Animal is eating
        myDog.bark(); // Output: Dog is barking

        System.out.println();

        // Multilevel inheritance example
        Labrador myLabrador = new Labrador();
        myLabrador.eat(); // Output: Animal is eating
        myLabrador.bark(); // Output: Dog is barking
        myLabrador.display(); // Output: Labrador is a type of dog
    }
}
```

5b) Write a program to illustrating all uses of super keywords.

- The **Animal** class has a **color** variable and a **display()** method that prints the color.
- The **Dog** class extends **Animal** and has its own **color** variable.
- In the **display()** method of **Dog**, **super.color** is used to access the **color** variable of the superclass.
- Also, **super.display()** is used to call the **display()** method of the superclass.
- In the **displayWithParameters()** method of **Dog**, **super.display()** is used to call the **display()** method of the superclass, and **super.display(message)** is used to call the overloaded **display()** method of the superclass.

```
class Animal {
    String color = "White";

    void display() {
        System.out.println("Animal color: " + color);
    }
}

class Dog extends Animal {
    String color = "Black";

    void display() {
        // Using super to access the color variable of the superclass
        System.out.println("Dog color: " + super.color);

        // Using super to call the display() method of the superclass
        super.display();
    }

    void displayWithParameters(String message) {
        // Using super to call the display() method of the superclass
        super.display();

        // Using super to call the overloaded display() method of the superclass
        super.display(message);
    }
}

public class SuperKeywordExample {
    public static void main(String[] args) {
        Dog myDog = new Dog();
        myDog.display();
        System.out.println();

        myDog.displayWithParameters("Hello");
    }
}
```

6a) Create an abstract class shape. Let rectangle and triangle inherit this shape class. Add necessary functions.

- We define an abstract class **Shape** with abstract methods **calculateArea()** and **calculatePerimeter()**.
- The **Rectangle** and **Triangle** classes inherit from **Shape** and provide implementations for these abstract methods.
- In the **main** method, we create objects of **Rectangle** and **Triangle** and calculate their area and perimeter.

```
// Abstract class Shape
abstract class Shape {
    // Abstract method to calculate area
    abstract double calculateArea();

    // Abstract method to calculate perimeter
    abstract double calculatePerimeter();
}

// Rectangle class inheriting from Shape
class Rectangle extends Shape {
    private double length;
    private double width;

    // Constructor
    public Rectangle(double length, double width) {
        this.length = length;
        this.width = width;
    }

    // Override calculateArea method
    @Override
    double calculateArea() {
        return length * width;
    }

    // Override calculatePerimeter method
    @Override
    double calculatePerimeter() {
        return 2 * (length + width);
    }
}

// Triangle class inheriting from Shape
class Triangle extends Shape {
    private double side1;
    private double side2;
    private double side3;

    // Constructor
    public Triangle(double side1, double side2, double side3) {
        this.side1 = side1;
```

```
        this.side2 = side2;
        this.side3 = side3;
    }

    // Override calculateArea method
    @Override
    double calculateArea() {
        // Using Heron's formula to calculate area of triangle
        double s = (side1 + side2 + side3) / 2;
        return Math.sqrt(s * (s - side1) * (s - side2) * (s - side3));
    }

    // Override calculatePerimeter method
    @Override
    double calculatePerimeter() {
        return side1 + side2 + side3;
    }
}

public class ShapeExample {
    public static void main(String[] args) {
        // Creating objects of Rectangle and Triangle
        Rectangle rectangle = new Rectangle(5, 4);
        Triangle triangle = new Triangle(3, 4, 5);

        // Calculating and printing area and perimeter of Rectangle
        System.out.println("Rectangle Area: " + rectangle.calculateArea());
        System.out.println("Rectangle Perimeter: " + rectangle.calculatePerimeter());

        // Calculating and printing area and perimeter of Triangle
        System.out.println("Triangle Area: " + triangle.calculateArea());
        System.out.println("Triangle Perimeter: " + triangle.calculatePerimeter());
    }
}
```

6b) Write a program to create a simple package.

Creating a simple package involves organizing your Java classes into a directory structure and adding a package declaration at the beginning of each class file. Here's a step-by-step guide:

1. **Create Directory Structure:** First, create a directory for your package. Let's name it **mypackage**.
2. **Write Java Classes:** Write your Java classes inside this directory. Each class should have a package declaration at the beginning. For example, if you're creating a package called **mypackage**, the package declaration should be **package mypackage;**.
3. **Compile Classes:** Compile your Java classes. Make sure your compiler is aware of the directory structure. You can do this by specifying the **-d** option followed by the directory name when compiling.
4. **Use the Package:** After compilation, you can use the classes in your package by importing them in other Java files.

Here's an example:

Let's say you want to create a package called **mypackage** with two classes: **Rectangle** and **Circle**.

Directory Structure:

```
mypackage/  
  Rectangle.java  
  Circle.java
```

Rectangle.java

```
package mypackage;  
  
public class Rectangle {  
    private double length;  
    private double width;  
  
    public Rectangle(double length, double width) {  
        this.length = length;  
        this.width = width;  
    }  
  
    public double calculateArea() {  
        return length * width;  
    }  
  
    public double calculatePerimeter() {  
        return 2 * (length + width);  
    }  
}
```

Circle.java

```
package mypackage;  
public class Circle {  
    private double radius;  
  
    public Circle(double radius) {  
        this.radius = radius;  
    }  
  
    public double calculateArea() {
```

```
        return Math.PI * radius * radius;
    }

    public double calculateCircumference() {
        return 2 * Math.PI * radius;
    }
}
```

Now, let's compile these classes. Navigate to the directory containing the mypackage directory and run:
javac mypackage/*.java

After compilation, you can use these classes in other Java files by importing them like this:

```
import mypackage.Rectangle;
import mypackage.Circle;
```

```
public class Main {
    public static void main(String[] args) {
        Rectangle rectangle = new Rectangle(5, 4);
        Circle circle = new Circle(3);

        System.out.println("Rectangle Area: " + rectangle.calculateArea());
        System.out.println("Circle Area: " + circle.calculateArea());
    }
}
```

Compile and run Main.java, and it will use classes from the mypackage package.

7 a) Implement an application that creates a Java interface.

// Define an interface called Shape

```
interface Shape {  
    // Abstract method to calculate area  
    double calculateArea();  
  
    // Abstract method to calculate perimeter  
    double calculatePerimeter();  
}
```

// Implementing classes of the Shape interface

class Rectangle implements Shape {

```
    private double length;  
    private double width;
```

// Constructor

```
    public Rectangle(double length, double width) {  
        this.length = length;  
        this.width = width;  
    }
```

// Implementing calculateArea method

```
    @Override  
    public double calculateArea() {  
        return length * width;  
    }
```

// Implementing calculatePerimeter method

```
    @Override  
    public double calculatePerimeter() {  
        return 2 * (length + width);  
    }
```

```
}
```

class Circle implements Shape {

```
    private double radius;
```

// Constructor

```
    public Circle(double radius) {  
        this.radius = radius;  
    }
```

// Implementing calculateArea method

```
    @Override  
    public double calculateArea() {  
        return Math.PI * radius * radius;  
    }
```

// Implementing calculatePerimeter method

```
    @Override  
    public double calculatePerimeter() {
```

```
        return 2 * Math.PI * radius;
    }
}
public class InterfaceExample {
    public static void main(String[] args) {
        // Creating objects of Rectangle and Circle
        Rectangle rectangle = new Rectangle(5, 4);
        Circle circle = new Circle(3);

        // Calculating and printing area and perimeter of Rectangle
        System.out.println("Rectangle Area: " + rectangle.calculateArea());
        System.out.println("Rectangle Perimeter: " + rectangle.calculatePerimeter());

        // Calculating and printing area and perimeter of Circle
        System.out.println("Circle Area: " + circle.calculateArea());
        System.out.println("Circle Perimeter: " + circle.calculatePerimeter());
    }
}
```

or

```
// Define a Java interface named Shape
interface Shape {
    // Abstract methods to calculate area and perimeter
    double calculateArea();
    double calculatePerimeter();
}

// Implement the interface in a class
class Circle implements Shape {
    private double radius;

    // Constructor
    public Circle(double radius) {
        this.radius = radius;
    }

    // Implement the abstract methods
    @Override
    public double calculateArea() {
        return Math.PI * radius * radius;
    }

    @Override
    public double calculatePerimeter() {
        return 2 * Math.PI * radius;
    }
}

public class InterfaceExample {
    public static void main(String[] args) {
        // Create an object of the Circle class
```

```
Circle circle = new Circle(3);
```

```
// Calculate and print area and perimeter
```

```
System.out.println("Circle Area: " + circle.calculateArea());
```

```
System.out.println("Circle Perimeter: " + circle.calculatePerimeter());
```

```
}
```

```
}
```

- We define a Java interface named **Shape** with two abstract methods: **calculateArea()** and **calculatePerimeter()**.
- The **Circle** class implements the **Shape** interface and provides implementations for the abstract methods.
- In the **main** method, we create an object of the **Circle** class, and we can use it to calculate the area and perimeter of the circle.

7b) Write an application that shows how to create a user-defined exception.

// Define a custom exception class

```
class MyCustomException extends Exception {  
    // Constructor with a message  
    public MyCustomException(String message) {  
        super(message);  
    }  
}
```

// Example application demonstrating the usage of the custom exception

```
public class CustomExceptionExample {  
    // A method that throws the custom exception  
    static void validateAge(int age) throws MyCustomException {  
        // Check if age is less than 18  
        if (age < 18) {  
            // If age is less than 18, throw the custom exception with a message  
            throw new MyCustomException("Age must be 18 or above.");  
        } else {  
            System.out.println("Valid age: " + age);  
        }  
    }  
  
    public static void main(String[] args) {  
        try {  
            // Call the method and pass an age  
            validateAge(20); // Valid age  
            validateAge(15); // This will throw the custom exception  
        } catch (MyCustomException e) {  
            // Catch the custom exception and print the error message  
            System.out.println("Caught Custom Exception: " + e.getMessage());  
        }  
    }  
}
```

- We define a custom exception class **MyCustomException** which extends **Exception**.
- The **validateAge()** method checks if the given age is less than 18. If it is, it throws **MyCustomException**.
- In the **main()** method, we call **validateAge()** with both valid and invalid ages to demonstrate how the custom exception works.
- When an invalid age is passed, the custom exception is thrown and caught in the **main()** method, and the error message is printed

8a) Create a customized exception and also make use of all the 5 exception keywords.

```
class CustomException(Exception):
```

```
    def __init__(self, message):  
        super().__init__(message)
```

```
# Example usage:
```

```
try:
```

```
    # Simulate some operation that might raise an exception  
    x = 10 / 0
```

```
except ZeroDivisionError:
```

```
    raise CustomException("Cannot divide by zero!") from None
```

```
except CustomException as e:
```

```
    print("Custom exception occurred:", e)
```

```
else:
```

```
    print("No exception occurred.")
```

```
finally:
```

```
    print("Finally block executed, regardless of exceptions.")
```

```
}
```

```
}
```

- **try** block: The code that might raise an exception is placed within this block.
- **except** block: Handles specific exceptions that might occur within the **try** block. Here, **ZeroDivisionError** is caught first, followed by **CustomException**.
- **raise** statement: Raises the **CustomException** with a custom error message if a **ZeroDivisionError** occurs.
- **finally** block: This block of code is always executed, regardless of whether an exception occurred or not. It's typically used for cleanup operations.
- **else** block: Executed if no exceptions were raised in the **try** block.

8b) Write an Applet that displays <Hello World= (Background color-black, text color-blue and your name in the statuswindow.)

```
import java.applet.*;
```

```
import java.awt.*;
```

```
public class HelloWorldApplet extends Applet {
```

```
    public void init() {  
        setBackground(Color.BLACK);  
        setForeground(Color.BLUE);  
    }
```

```
    public void paint(Graphics g) {  
        g.drawString("Hello World", 20, 20);  
    }
```

```
    public void start() {  
        showStatus("Your Name Here");  
    }  
}
```

```
<html>
```

```
<head>
```

```
    <title>Hello World Applet</title>
```

```
</head>
```

```
<body>
```

```
    <applet code="HelloWorldApplet.class" width="200" height="50">
```

```
        Your browser does not support Java applets.
```

```
    </applet>
```

```
</body>
```

```
</html>
```

9a) Write a java program to show multithreaded producer and consumer application.

import java.util.LinkedList;

```
class ProducerConsumer {
    private LinkedList<Integer> buffer = new LinkedList<>();
    private final int CAPACITY = 5;

    public void produce() throws InterruptedException {
        int value = 0;
        while (true) {
            synchronized (this) {
                while (buffer.size() == CAPACITY) {
                    wait();
                }
                System.out.println("Producer produced: " + value);
                buffer.add(value++);
                notify();
                Thread.sleep(1000);
            }
        }
    }
}
```

```
    public void consume() throws InterruptedException {
        while (true) {
            synchronized (this) {
                while (buffer.size() == 0) {
                    wait();
                }
                int val = buffer.removeFirst();
                System.out.println("Consumer consumed: " + val);
                notify();
                Thread.sleep(1000);
            }
        }
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        ProducerConsumer pc = new ProducerConsumer();

        Thread producerThread = new Thread(() -> {
            try {
                pc.produce();
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        });

        Thread consumerThread = new Thread(() -> {
            try {
```

```
        pc.consume();
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
});

producerThread.start();
consumerThread.start();
}
}
```


9b) Write a program that displays your name whenever the mouse is clicked.

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class NameDisplay extends JFrame implements MouseListener {

    private JLabel nameLabel;

    public NameDisplay() {
        setTitle("Name Display");
        setSize(300, 200);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        nameLabel = new JLabel("Your Name");
        nameLabel.setHorizontalAlignment(JLabel.CENTER);
        nameLabel.setFont(new Font("Arial", Font.BOLD, 20));
        add(nameLabel, BorderLayout.CENTER);

        addMouseListener(this);
    }

    public void mouseClicked(MouseEvent e) {
        nameLabel.setText("Your Name");
    }

    public void mouseEntered(MouseEvent e) {}

    public void mouseExited(MouseEvent e) {}

    public void mousePressed(MouseEvent e) {}

    public void mouseReleased(MouseEvent e) {}

    public static void main(String[] args) {
        SwingUtilities.invokeLater(new Runnable() {
            public void run() {
                new NameDisplay().setVisible(true);
            }
        });
    }
}
```

10a) Write a program that read from a file and writes to another file.

import java.io.*;

```
public class FileReadWrite {
    public static void main(String[] args) {
        String inputFile = "input.txt";
        String outputFile = "output.txt";

        try {
            FileReader fileReader = new FileReader(inputFile);
            BufferedReader bufferedReader = new BufferedReader(fileReader);

            FileWriter fileWriter = new FileWriter(outputFile);
            BufferedWriter bufferedWriter = new BufferedWriter(fileWriter);

            String line;
            while ((line = bufferedReader.readLine()) != null) {
                bufferedWriter.write(line);
                bufferedWriter.newLine(); // Add newline after each line
            }

            // Close readers and writers
            bufferedReader.close();
            bufferedWriter.close();

            System.out.println("File successfully read from " + inputFile + " and written to " + outputFile +
                ".");
        } catch (FileNotFoundException e) {
            System.out.println("File not found: " + e.getMessage());
        } catch (IOException e) {
            System.out.println("Error reading/writing file: " + e.getMessage());
        }
    }
}
```

In this program:

- We specify the input and output file paths (**input.txt** and **output.txt** respectively).
- We use **FileReader** and **BufferedReader** to read from the input file line by line.
- We use **FileWriter** and **BufferedWriter** to write each line to the output file.
- The **readLine()** method returns **null** when the end of the file is reached.
- We close the readers and writers using the **close()** method to release resources after use.

Ensure that the **input.txt** file exists and contains some text. When you run this program, it will read the content of **input.txt** and write it to **output.txt**. If **input.txt** contains:

10b) Create a simple login table and establish JDBC connection using java program.

Create a MySQL Database and Table:

1. First, you need to create a MySQL database and a table to store user login information. Here's an example SQL script to create the database and table:

```
CREATE DATABASE IF NOT EXISTS mydb;
```

```
USE mydb;
```

```
CREATE TABLE IF NOT EXISTS users (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    username VARCHAR(50) NOT NULL,  
    password VARCHAR(50) NOT NULL  
);
```

2. Java Program to Establish JDBC Connection: Once you have the database and table set up, you can write a Java program to establish a JDBC connection and perform operations on the table. Below is an example Java program:

```
import java.sql.*;
```

```
public class JDBCExample {  
    // JDBC URL, username, and password of MySQL server  
    static final String JDBC_URL = "jdbc:mysql://localhost:3306/mydb";  
    static final String USERNAME = "your_username";  
    static final String PASSWORD = "your_password";  
  
    public static void main(String[] args) {  
        Connection conn = null;  
        Statement stmt = null;  
  
        try {  
            // Register JDBC driver  
            Class.forName("com.mysql.cj.jdbc.Driver");  
  
            // Open a connection  
            System.out.println("Connecting to database...");  
            conn = DriverManager.getConnection(JDBC_URL, USERNAME, PASSWORD);  
  
            // Execute a query  
            System.out.println("Creating statement...");  
            stmt = conn.createStatement();  
            String sql = "SELECT * FROM users";  
            ResultSet rs = stmt.executeQuery(sql);  
  
            // Display the result set  
            while (rs.next()) {  
                int id = rs.getInt("id");  
                String username = rs.getString("username");  
                String password = rs.getString("password");  
  
                System.out.println("ID: " + id + ", Username: " + username + ", Password: " + password);  
            }  
        }  
    }  
}
```

```
        // Clean-up environment
        rs.close();
        stmt.close();
        conn.close();
    } catch (SQLException se) {
        // Handle errors for JDBC
        se.printStackTrace();
    } catch (Exception e) {
        // Handle errors for Class.forName
        e.printStackTrace();
    } finally {
        // Finally block used to close resources
        try {
            if (stmt != null) stmt.close();
        } catch (SQLException se2) {
        } // nothing we can do
        try {
            if (conn != null) conn.close();
        } catch (SQLException se) {
            se.printStackTrace();
        } // end finally try
    } // end try
    System.out.println("Goodbye!");
}
```

11a) Create runnable jar file in java.

To create a runnable JAR file in Java, you need to follow these steps:

1. **Compile your Java code:** Make sure you have compiled your Java code into **.class** files.
2. **Create a manifest file:** You need to create a manifest file (**Manifest.txt**) that specifies the main class. This file should be placed in the same directory as your compiled **.class** files.

Example **Manifest.txt**:

Main-Class: com.example.MainClassName

Replace **com.example.MainClassName** with the fully qualified name of your main class.

3. **Create the JAR file:** Use the **jar** command-line tool provided with the JDK to create the JAR file. Run the following command in the terminal:

```
jar cfm YourJarName.jar Manifest.txt -C path/to/compiled/classes .
```

Replace **YourJarName.jar** with the desired name of your JAR file and **path/to/compiled/classes** with the path to the directory containing your compiled **.class** files.

This command creates a JAR file (**YourJarName.jar**) with the specified manifest file and includes all the compiled **.class** files.

4. **Test the JAR file:** You can test the generated JAR file by running it with the **java** command:

```
java -jar YourJarName.jar
```

Replace **YourJarName.jar** with the name of your JAR file.

Once you created a runnable JAR file in Java. Now you can distribute this JAR file and run it on any machine with Java installed.

```
public class SumCalculator {
    public static void main(String[] args) {
        if (args.length != 2) {
            System.out.println("Usage: java SumCalculator <number1> <number2>");
            return;
        }

        try {
            double num1 = Double.parseDouble(args[0]);
            double num2 = Double.parseDouble(args[1]);
            double sum = num1 + num2;
            System.out.println("Sum: " + sum);
        } catch (NumberFormatException e) {
            System.out.println("Invalid input. Please enter valid numbers.");
        }
    }
}
```

This program takes two numbers as command-line arguments and calculates their sum.

To create a runnable JAR file for this program, follow these steps:

1. Compile the Java code (**SumCalculator.java**) to generate the **.class** file:

```
javac SumCalculator.java
```

2. Create a manifest file (**Manifest.txt**) specifying the main class (**SumCalculator**) with the following content:

Main-Class: SumCalculator

3. Create the JAR file using the **jar** command:

This command creates a JAR file named SumCalculator.jar containing the compiled class file (SumCalculator.class) and using the specified manifest file (Manifest.txt).

4. Run the JAR file with command-line arguments to calculate the sum of two numbers:

```
java -jar SumCalculator.jar 10 20
```

Replace **10** and **20** with your desired numbers.

11b) Display image on a button in swing.

```
import javax.swing.*;
import java.awt.*;

public class ImageButtonExample {
    public static void main(String[] args) {
        // Create a JFrame
        JFrame frame = new JFrame("Image Button Example");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setSize(300, 200);

        // Create an ImageIcon
        ImageIcon icon = new ImageIcon("image.png"); // Change "image.png" to the path of your
        image file

        // Create a JButton with the ImageIcon
        JButton button = new JButton(icon);

        // Add the button to the frame
        frame.getContentPane().add(button, BorderLayout.CENTER);

        // Display the frame
        frame.setVisible(true);
    }
}
```

In this example:

- Replace "**image.png**" with the path to your image file.
- An **ImageIcon** is created using the specified image file.
- A **JButton** is created with the **ImageIcon** as its icon.
- The button is added to the frame's content pane.
- Finally, the frame is set visible.

12a) Write a servlet which displays current system date and time.

```
import java.io.IOException;
import java.io.PrintWriter;
import java.util.Date;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

@WebServlet("/DateTimeServlet")
public class DateTimeServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
        response.setContentType("text/html");

        PrintWriter out = response.getWriter();
        out.println("<html>");
        out.println("<head>");
        out.println("<title>Current Date and Time</title>");
        out.println("</head>");
        out.println("<body>");
        out.println("<h2>Current Date and Time</h2>");
        out.println("<p>");
        out.println("Current Date and Time: " + new Date());
        out.println("</p>");
        out.println("</body>");
        out.println("</html>");
    }
}
```

In this servlet:

- We import necessary classes for servlet functionality.
- We annotate the servlet class with **@WebServlet("/DateTimeServlet")** to specify the URL pattern at which this servlet will be accessible.
- In the **doGet()** method, we set the content type of the response to "text/html".
- We retrieve a **PrintWriter** object from the response to write HTML content.
- We construct a simple HTML response that displays the current date and time using the **Date** class.
- The response is then sent to the client.

Make sure to deploy this servlet in a servlet container such as Apache Tomcat. When you access the URL mapped to this servlet (e.g., **http://localhost:8080/your-web-app/DateTimeServlet**), it will display the current date and time.

12b) Write a program that handle HTTP request and HTTP response.

```
import com.sun.net.httpserver.HttpServer;
import com.sun.net.httpserver.HttpHandler;
import com.sun.net.httpserver.HttpExchange;
import java.io.IOException;
import java.io.OutputStream;

public class HttpServerExample {
    public static void main(String[] args) throws IOException {
        // Create HTTP server on port 8000
        HttpServer server = HttpServer.create(new InetSocketAddress(8000), 0);

        // Create a context for the "/" path
        server.createContext("/", new MyHandler());

        // Start the server
        server.start();
        System.out.println("Server started on port 8000");
    }

    static class MyHandler implements HttpHandler {
        @Override
        public void handle(HttpExchange exchange) throws IOException {
            // Set the response headers
            exchange.getResponseHeaders().set("Content-Type", "text/plain");

            // Send the response content
            String response = "Hello, this is the HTTP server!";
            exchange.sendResponseHeaders(200, response.length());
            OutputStream os = exchange.getResponseBody();
            os.write(response.getBytes());
            os.close();
        }
    }
}
```

In this program:

- We import necessary classes for handling HTTP requests and responses.
- We create an **HttpServer** instance listening on port 8000.
- We create a context for the root path "/" and assign a **MyHandler** instance to handle incoming HTTP requests.
- Inside the **MyHandler** class, we implement the **handle** method to process HTTP requests and send HTTP responses.
- In the **handle** method, we set the response headers and send the response content.

To run this program, make sure you have the required dependencies in your classpath. You can compile and run the program using the following commands:

```
javac HttpServerExample.java
java HttpServerExample
```

Once the program is running, you can access the HTTP server by navigating to **http://localhost:8000** in your web browser. You should see the response "Hello, this is the HTTP server!" displayed in the browser.

Model Viva :

Q1- What is The Java Features ?

A1-1.Compiled and Interpreted 2.Platform-Independent and Portable 3. Object – Oriented
4.Robust and Secure 5.Distributed 6.Simple, Small and Familiar 7.Multithreaded and interactive
8.High Performance9.Dynamic and Extensible

Q2 -How Java Differs From C ?

A2-1.Java does not include the C unique statement keywords goto, size of, and type def.

2.Java does not contain the data type struct,union and enum.3.Java does not define the type modifiers keywords auto,extern ,register,signed,and unsigned.4.Java does not support an explicit pointer type.

Q3- How Java Differs From C ++ ?

A3-1.java does not support operator overloading.2. Java does not have template classes as in C++.3.Java does not multiple inheritance of classes.This is accomplished using a new feature called “interface”.

Q4- What is The Java Components?

A4-1.Object and Classes 2.Data Abstraction and Encapsulation 3.Inheritance 4.Polymorphism
5.Dynamic Binding 6.Message Communication.

Q5- What is the Java Development kit (JDK) ?

A5-1.appletviewer(for viewing Java applets) 2.javac(Java compiler) 3.java(Java interpreter)
4.javap(Java disassembler) 5.javah(for C header files) 6.javadoc(for creating HTML documents)
7.jdb(Java debugger)

Q6- What is the Variables ?

A6- A variable is an identifier that denotes a storage location used to store a data value.

Q7- What are Class?

A7- Class is a template for multiple objects with similar features and it is a blue print for objects.
It defines a type of object according to the data the object can hold and the operations the object can perform.

Q8- What are Primitive data types?

A8- Primitive data types are 8 types and they are: byte, short, int, long, float, double, boolean, char.

Q9- What is the method overloading.?

A9- Function with same name but different argument perform different task known as method overloading.

Q10- What are inner class and anonymous class?

A10- Inner class : classes defined in other classes, including those defined in methods are called innerclasses. An inner class can have any accessibility including private. Anonymous class : Anonymous class is a class defined inside a method without a name and is instantiated and declared in the same place and cannot have explicit constructors.

Title of the Practical: Program to design comparison two Variable use if statements.

Q1-What is the if statement ?

A1- The if statement is a powerful decision making statement and is used to control the flow of execution of statements.

Q2- What is the If-else statement ?

A2- The if.....else statement is an extension of the simple if statement.

Q3- What is the Switch-case statement ?

A3- The switch statement tests the value of a given variable(or expression) against a list of case values and when a match is found , a block of statements associated with that case is executed.

Q4- What are the Break ?

A4- The break statement is used to terminate a statement sequence.

Q5-What are Continue ?

A5- The continue statement causes control to be transferred directly to the conditional

expression that controls the loop.

Q6-What is the While loop ?

A6- The while loop is Java's most fundamental looping statement. It repeats a statement or block while its controlling expression is true.

Q7-What is the Do – while loop ?

A7- The do –while is an exit – controlled loop. Based on a condition, the control is transferred back to a particular point in the program.

Q8-What is the for loop ?

A8- The for is an entry-controlled loop and is used when an action is to be repeated for a predetermined number of times.

Q9-What is the Operators ?

A9- Operators are used in programs to manipulate data and variables.

Q10-What is the Arithmetic Expressions ?

A10- An arithmetic expression is a combination of variables, constants, and operators arranged as per the syntax of the language.

Title of the Practical: Program to design Constructor.

Q1- What is the Constructor ?

A2- Constructor is a special member function of class call automatically when object is created.

Q2- What is the Parameterized constructors ?

A2- The constructors that can take argument are called parameterized constructors.

Q3- What is OOPs?

A3- Object oriented programming organizes a program around its data, i. e., objects and a set of well defined interfaces to that data. An object-oriented program can be characterized as data controlling access to code.

Q4- What are Encapsulation?

A4- Encapsulation is the mechanism that binds together code and data it manipulates and keeps both safe from outside interference and misuse.

Q5- What are Polymorphism?

A5- Polymorphism is the feature that allows one interface to be used for general class actions.

Q6- What is the difference between procedural and object-oriented programs?

A6- a) In procedural program, programming logic follows certain procedures and the instructions are executed one after another. In OOP program, unit of program is object, which is nothing but combination of data and code. b) In procedural program, data is exposed to the whole program whereas in OOPs program, it is accessible within the object and which in turn assures the security of the code.

Q7- What is an Object and how do you allocate memory to it?

A7- Object is an instance of a class and it is a software unit that combines a structured set of data with a set of operations for inspecting and manipulating that data. When an object is created using new operator, memory is allocated to it.

Q8- What is the difference between constructor and method?

A8- Constructor will be automatically invoked when an object is created whereas method has to be called explicitly.

Q9- What are methods and how are they defined?

A9- Methods are functions that operate on instances of classes in which they are defined.

Q10- What is the use of bin and lib in JDK?

A10- Bin contains all tools such as javac, appletviewer, awt tool, etc., whereas lib contains API and all packages.

Title of the Practical: Program to design toUpperCase() and toLowerCase Type Conversion.

Q1- What is casting?

A1- Casting is used to convert the value of one type to another.

Q2- How many ways can an argument be passed to a subroutine and explain them?

A2- An argument can be passed in two ways. They are passing by value and passing by reference. Passing by value: This method copies the value of an argument into the formal parameter of the subroutine. Passing by reference: In this method, a reference to an argument (not the value of the argument) is passed to the parameter.

Q3- What is the difference between an argument and a parameter?

A3- While defining method, variables passed in the method are called parameters. While using those methods, values passed to those variables are called arguments.

Q4- What are different types of access modifiers?

A4- public: Any thing declared as public can be accessed from anywhere. private: Any thing declared as private can't be seen outside of its class. protected: Any thing declared as protected can be accessed by classes in the same package and subclasses in the other packages. default modifier : Can be accessed only to classes in the same package.

Q5- What is UNICODE?

A5- Unicode is used for internal representation of characters and strings and it uses 16 bits to represent each other.

Q6- What is the difference between String and String Buffer?

A6- a) String objects are constants and immutable whereas StringBuffer objects are not. b) String class supports constant strings whereas StringBuffer class supports growable and modifiable strings.

Q7- What is the difference between Array and vector?

A7- Array is a set of related data type and static whereas vector is a growable array of objects and dynamic.

Q8- What are wrapper classes?

A8- Wrapper classes are classes that allow primitive types to be accessed as objects.

Q9- What are Vector?

A9- Vector : The Vector class provides the capability to implement a growable array of objects.

Q10- What is a stream ?

A10- A Stream is an abstraction that either produces or consumes information.

Title of the Practical: Programs to design Single Inheritance.

Q10- What is the Inheritance?

A10- Inheritance is the process by which object of one class acquire the properties of object of another class.

Q2- What is the Package?

A2- Package is the collection of class stored in a folder. These class can be used in any java program.

Q3- What is meant by Inheritance and what are its advantages?

A3- Inheritance is the process of inheriting all the features from a class. The advantages of inheritance are reusability of code and accessibility of variables and methods of the super class by subclasses.

Q4- What is an abstract class?

A4- An abstract class is a class designed with implementation gaps for subclasses to fill in and is deliberately incomplete.

Q5- What is a clone able interface and how many methods does it contain?

A5- It is not having any method because it is a TAGGED or MARKER interface.

Q6- What is the difference between abstract class and interface?

A6- a) All the methods declared inside an interface are abstract whereas abstract class must have at least one abstract method and others may be concrete or abstract. b) In abstract class, key word abstract must be used for the methods whereas interface we need not use that keyword for the methods. c) Abstract class must have subclasses whereas interface can't have subclasses.

Q7- Can you have an inner class inside a method and what variables can you access?

A7- Yes, we can have an inner class inside a method and final variables can be accessed.

Q8- What is the class and interface in java to create thread and which is the most advantageous method?

A8- Thread class and Runnable interface can be used to create threads and using Runnable interface is the most advantageous method to create threads because we need not extend thread class here.

Q9- What are the states associated in the thread?

A9- Thread contains ready, running, waiting and dead states.

Q10- What is deadlock?

A10- When two threads are waiting each other and can't precede the program is said to be deadlock.

Title of the Practical: Programs to design interface.

Q1- What is the Interfaces?

A1- Using the keyword interface ,you can fully abstract a class' interface from its implementation.

Q2-What is The Try-Catch-Finally ?

A2- 1.Java uses a keyword Try to handle a run-time error, simply enclose the code that you want to monitor inside a try block. Immediately following the try block. 2.include a catch clause that specifies the exception type that you wish to catch. 3.finally block can be used to handle any exception generated within a try block. 4.When a finally block is defined , this is guaranteed to execute ,regardless of whether or not an exception is thrown.

Q3-What are the Throw statements ?

A3- you have only been catching exceptions that are thrown by the Java run-time system.

Q4-What are the Throws ?

A4- If a method is capable of causing an exception that it does not handle, it must specify this behavior so that callers of the method can guard themselves against that exception.

Q5- What is Java?

A5- Java is an object-oriented programming language developed initially by James Gosling and colleagues at Sun Microsystems.

Q6- Can you have virtual functions in Java?

A6- Yes, all functions in Java are virtual by default.

Q7-What is the NullPointerException ?

A7- When an object is not initialized, the default value is null.

Q8- Name the containers which uses Border Layout as their default layout?

A8- Containers which uses Border Layout as their default are: window, Frame and Dialog classes.

Q9- What is Collection API ?

A9- The Collection API is a set of classes and interfaces that support operation on collections of objects.

Q10- What is a transient variable in Java?

A10- A transient variable is a variable that may not be serialized. If you don't want some field to be serialized, you can mark that field transient or static.

Title of the Practical: Program to design exception by use Try-Catch-Finally.

Q1-. What is the exception ?

A1- An exception is a condition that is caused by a run-time error in the program.

Q2- What is the difference between an argument and a parameter?

A2- While defining method, variables passed in the method are called parameters. While using those methods, values passed to those variables are called arguments.

Q3- What are different types of access modifiers?

A3- public: Any thing declared as public can be accessed from anywhere. private: Any thing declared as private can't be seen outside of its class. protected: Any thing declared as protected

can be accessed by classes in the same package and subclasses in the other packages. default modifier : Can be accessed only to classes in the same package.

Q4- What is Garbage Collection and how to call it explicitly?

A4- When an object is no longer referred to by any variable, java automatically reclaims memory used by that object. This is known as garbage collection. System. gc() method may be used to call it explicitly.

Q5- What is finalize() method?

A5-finalize () method is used just before an object is destroyed and can be called just prior to garbage collection.

Q6- What are Transient and Volatile Modifiers?

A6- Transient: The transient modifier applies to variables only and it is not stored as part of its object's Persistent state. Transient variables are not serialized. Volatile: Volatile modifier applies to variables only and it tells the compiler that the variable modified by volatile can be changed unexpectedly by other parts of the program.

Q7- What is the difference between this() and super()?

A7- this() can be used to invoke a constructor of the same class whereas super() can be used to invoke a super class constructor.

Q8- What is the difference between superclass and subclass?

A8- A super class is a class that is inherited whereas sub class is a class that does the inheriting.

Q9- What modifiers may be used with top-level class?

A9- public, abstract and final can be used for top-level class.

Q10- What is the difference between exception and error?

A10- The exception class defines mild error conditions that your program encounters. Exceptions can occur when trying to open the file, which does not exist, the network connection is disrupted, operands being manipulated are out of prescribed ranges, the class file you are interested in loading is missing. The error class defines serious error conditions that you should not attempt to recover from. In most cases it is advisable to let the program terminate when such an error is encountered.

Title of the Practical: Program to design multi-threading.

Q1- What is the multithreaded?

A1- A multithreaded program contains two or more parts that can run concurrently .Each part of such a program is called a thread , and each thread define a separate path of execution.

Q2- What is the thread ?

A2- A thread is similar to a program that has a single flow of control.It has a beginning,a body,and an end ,and executes commands sequentially.

Q3- What is the Inter thread communication ?

A3- Java includes an elegant interprocess communication mechanism via the wait(), notify(), and notifyAll() methods.

Q4- What is the Threads synchronization ?

A4- When two or more threads need access to a shared resource, they need some way to ensure that the resource will be used by only one thread at a time. The process by which this is achieved is called synchronization.

Q5- What is the synchronized Methods ?

A5- There are Three synchronized Methods : The call() method, run() method,and sleep() method.

Q6- What is the difference between process and thread?

A6- Process is a program in execution whereas thread is a separate path of execution in a program.

Q7- What are synchronized methods and synchronized statements?

A7- Synchronized methods are methods that are used to control access to a method or an object.

A thread only executes a synchronized method after it has acquired the lock for the method's object or class. Synchronized statements are similar to synchronized methods. A synchronized statement can only be executed after a thread has acquired the lock for the object or class referenced in the synchronized statement.

Q8- What are three ways in which a thread can enter the waiting state?

A8- A thread can enter the waiting state by invoking its `sleep()` method, by blocking on IO, by unsuccessfully attempting to acquire an object's lock, or by invoking an object's `wait()` method. It can also enter the waiting state by invoking its (deprecated) `suspend()` method.

Q9- Can a lock be acquired on a class?

A9- Yes, a lock can be acquired on a class. This lock is acquired on the class's `Class` object.

Q10- What is the preferred size of a component?

A10- The preferred size of a component is the minimum component size that will allow the component to display normally.

Title of the Practical: Program to design Applet skeleton.

Q1- What is the difference between an applet and a servlet?

A1- a) Servlets are to servers what applets are to browsers. b) Applets must have graphical user interfaces whereas servlets have no graphical user interfaces.

Q2- What is Inet address?

A2- Every computer connected to a network has an IP address. An IP address is a number that uniquely identifies each computer on the Net. An IP address is a 32-bit number.

Q3- What is an event and what are the models available for event handling?

A3- An event is an event object that describes a state of change in a source. In other words, event occurs when an action is generated, like pressing button, clicking mouse, selecting a list, etc. There are two types of models for handling events and they are: a) event-inheritance model and b) event-delegation model

Q4- When do you use codebase in applet?

A4- When the applet class file is not in the same directory, codebase is used.

Q5- What is the difference between Assignment and Initialization?

A5- Assignment can be done as many times as desired whereas initialization can be done only once.

Q6- What is source and listener?

A6- source : A source is an object that generates an event. This occurs when the internal state of that object changes in some way. listener : A listener is an object that is notified when an event occurs. It has two major requirements. First, it must have been registered with one or more sources to receive notifications about specific types of events. Second, it must implement methods to receive and process these notifications.

Q7- What is adapter class?

A7- An adapter class provides an empty implementation of all methods in an event listener interface

Q8- What is meant by controls and what are different types of controls in AWT?

A8- Controls are components that allow a user to interact with your application and the AWT supports the following types of controls: Labels, Push Buttons, Check Boxes, Choice Lists, Lists, Scrollbars, Text Components. These controls are subclasses of `Component`.

Q9- What is servlet?

A9- Servlets are modules that extend request/response-oriented servers, such as java-enabled web servers. For example, a servlet might be responsible for taking data in an HTML order-entry form and applying the business logic used to update a company's order database.

Q10- What is URL?

A10- URL stands for Uniform Resource Locator and it points to resource files on the Internet. URL has four components: `http://www.address.com:80/index.html`, where `http` - protocol name, `address` - IP address or host name, `80` - port number and `index.html` - file path.